VERIFICATION OF FIFO

Project 1

Saeed Maher Saeed

Table of Contents

The verification plan:

Label	Design requirement describtion	Stimulus Generation	Functional coverage	Fuction check
FIFO1	When the reset is asserted, all the outputs should be low except data out will be the same as prevoius cycle	Directed at the start of the simulation, then randomized in the simulation	-	Immediate assertion implemented in the top module to check the ascynchronous reset for outputs
FIFO2	When the reset is asserted, all the outputs and the internal signals of the design should be low except data out will be the same as prevoius cycle	Directed at the start of the simulation, then randomized in the simulation	-	Immediate assertion implemented in the design module to check the ascynchronous reset for outputs and internal signals
FIFO3	when the reset is not active and the write enable is active, the data should be written in the FIFO if the FIFO is not full in same order of the the wrtitting procces and after write it the wr_ack signal should be high	Randomization under constraints that reset is not active most of the time, the write enable is active by default value perecentage called "WR_EN_ON_DIST" and its default value is 70 and that is what used in this verification	Cover the write enable, read enable and write ack signals then cross cover them.	Concurrent assertion in the design module to check the write ack signal.
FIFO4	when the reset is not active and the write enable is active for full FIFO it should ignore the data given, the wr_ack signal should be low and the overflow flag to be high	Randomization under constraints that reset is not active most of the time, the write enable is active by default value perecentage called "WR_EN_ON_DIST" and its default value is 70 and that is what used in this verification	Cover the write enable, read enable and overflow signals then cross cover them.	Concurrent assertion in the design module to check the overflow signal.
FIFO5	when the reset is not active and the read enable signal is active for an empty FIFO, the underflow signal should be high	Randomization under constraints that reset is not active most of the time, the read enable is active by default value perecentage called "RD_EN_ON_DIST" and its default value is 30 and that is what used in this verification	Cover the write enable, read enable and underflow signals then cross cover them.	Concurrent assertion in the design module to check the underflow signal
FIFO6	when the FIFO is empty, the empty flag should be high and any read operation is ignored.	Randomization under constraint that reset is not active most of the time.	When reset is not active, cover the write enable, read enable and empty flag signals then cross cover them with ignoring the bins when empty is high and at same time write enable is high.	Immediate assertion in the design module to check the empty flag signal
FIFO7	when the FIFO is full, the full flag should be high and any write operation should be ignored.	Randomization under constraint that reset is not active most of the time.	When reset is not active, cover the write enable, read enable and full flag signals then cross cover them with ignoring the bins when full is high and at same time read enable is high.	Immediate assertion in the design module to check the full flag signal
FIFO8	when the FIFO has only one place empty, the almostfull flag should be high.	Randomization under constraint that reset is not active most of the time.	When reset is not active, cover the write enable, read enable and almostfull flag signals then cross cover them.	Immediate assertion in the design module to check the almostfull flag signal
FIFO9	when the FIFO has only one place written in it, the almostempty flag should be high.	Randomization under constraint that reset is not active most of the time.	When reset is not active, cover the write enable, read enable and almostempty flag signals then cross cover them.	Immediate assertion in the design module to check the almostempty flag signal
FIFO10	when the reset is not active and the FIFO is not full, for the write pointer if it reached it boundry "max value (7)" for next write operation it should return to zero.	Randomization under constraints that reset is not active most of the time, the write enable is active by default value perecentage called "WR_EN_ON_DIST" and its default value is 70 and that is what used in this verification	cover the write enable and read enable signals	Concurrent assertion in the design module to check the wraparound functionality of the internal write pointer
FIFO11	when the reset is not active and the FIFO is not empty, for the read pointer if it reached it boundry "max value (7)" for next read operation it should return to zero.	Randomization under constraints that reset is not active most of the time, the read enable is active by default value perecentage called "RD_EN_ON_DIST" and its default value is 30 and that is what used in this verification	cover the write enable and read enable signals	Concurrent assertion in the design module to check the wraparound functionality of the internal read pointer
FIFO12	when the reset is not active, it is expected that the write pointer value will never exceed the value of (FIFO_depth-1)	Randomization under constraints that reset is not active most of the time, the write enable is active by default value perecentage called "WR_EN_ON_DIST" and its default value is 70 and that is what used in this verification	cover the write enable and read enable signals	Concurrent assertion in the design module to check the limitation of the internal read pointer
FIFO13	when the reset is not active, it is expected that the read pointer value will never exceed the value of (FIFO_depth-1)	Randomization under constraints that reset is not active most of the time, the read enable is active by default value perecentage called "RD_EN_ON_DIST" and its default value is 30 and that is what used in this verification	cover the write enable and read enable signals	Concurrent assertion in the design module to check the limitation of the internal write pointer
FIFO14	when the reset is not active, it is expected that the ineranal counter signal value will never exceed the value of (FIFO_depth)	Randomization under constraints that reset is not active most of the time, the write enable is active by default value perecentage called "WR_EN_ON_DIST" and its default value is 70 and that is what used in this verification while the read enable is active by default value perecentage called "RD_EN_ON_DIST" and its default value is 30 and that is what used in this verification	cover the write enable and read enable signals	Concurrent assertion in the design module to check the limitation of the internal counter signal
FIFO15	when the reset is not active when read enable is high. It is expected that the data out will get the data that has the turn in the FIFO unless the FIFO is empty then the read operation is ignored and the data out will keep its value as it is	Randomization under constraints that reset is not active most of the time, the read enable is active by default value perecentage called "RD_EN_ON_DIST" and its default value is 30 and that is what used in this verification	cover the write enable and read enable signals	output checked against refernce model task created in the score board package.

Here is a link of the full verification plan: Link.

The system Verilog codes:

➢ The design code:

Note: the design code changed to be (.sv) file and have some modifications as it was buggy.

The modifications:

- The design wasn't handling the cases that if FIFO is empty or full and has at same time high write and read enables so I added when full and both are high to decrement the count and when empty to increment the count.
- The underflow signal was defined as combinational signal so I redefined it inside the always block as a sequential signal.
- Some signals are forgotten to be reseted when reset asserted like: wr_ack and overflow
- Overflow and underflow to be zero when read and write to avoid the problem that in cycle it read to be overflow then next cycle get both high wr_en and rd_en will still make the overflow 1 and then for next cycle the wr_en is high will write correctly and make the overflow still 1 so to avoid it I accessed them to be zero. The design will still be synthesizable as it accessd at same always block with different conds of if so it's like a MUX.

The code:

```
////////////////////////////////////////////////////////////////////////////////
// Author: Kareem Waseem
// Course: Digital Verification using SV & UVM
//
// Description: FIFO Design
//
////////////////////////////////////////////////////////////////////////////////
module FIFO(FIFO_interface.DUT F_if);

localparam max_fifo_addr = $clog2(F_if.FIFO_DEPTH);
reg [F_if.FIFO_WIDTH-1:0] mem [F_if.FIFO_DEPTH-1:0];
reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
reg [max_fifo_addr:0] count;

always @(posedge F_if.clk or negedge F_if.rst_n) begin
    if (!F_if.rst_n) begin
```

```
        wr_ptr <= 0;
        F_if.wr_ack <= 0;
        F_if.overflow <= 0;
    end
    else if (F_if.wr_en && count < F_if.FIFO_DEPTH) begin
        mem[wr_ptr] <= F_if.data_in;
        F_if.wr_ack <= 1;
        wr_ptr <= wr_ptr + 1;
        F_if.overflow <= 0;
    end
    else begin
        F_if.wr_ack <= 0;
        if (F_if.full && F_if.wr_en)
            F_if.overflow <= 1;
        else
            F_if.overflow <= 0;
    end
end

always @(posedge F_if.clk or negedge F_if.rst_n) begin
    if (!F_if.rst_n) begin
        rd_ptr <= 0;
        F_if.underflow <= 0;
    end
    else if (F_if.rd_en && count != 0) begin
        F_if.data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;
        F_if.underflow <= 0;
    end
    else begin
        if (F_if.empty && F_if.rd_en) begin
            F_if.underflow <= 1;
        end
        else begin
            F_if.underflow <= 0;
        end
    end
end

always @(posedge F_if.clk or negedge F_if.rst_n) begin
    if (!F_if.rst_n) begin
        count <= 0;
    end
    else begin
        if  ( ({F_if.wr_en, F_if.rd_en} == 2'b10) && !F_if.full) begin
```

```
            count <= count + 1;
        end
        else if ( ({F_if.wr_en, F_if.rd_en} == 2'b01) && !F_if.empty) begin
            count <= count - 1;
        end
        else if (({F_if.wr_en, F_if.rd_en} == 2'b11) && F_if.empty) begin
            count <= count + 1;
        end
        else if ( ({F_if.wr_en, F_if.rd_en} == 2'b11) && F_if.full) begin
            count <= count - 1;
        end
    end
end

assign F_if.full = (count == F_if.FIFO_DEPTH)? 1 : 0;
assign F_if.empty = (count == 0)? 1 : 0;
assign F_if.almostfull = (count == F_if.FIFO_DEPTH-1)? 1 : 0;
assign F_if.almostempty = (count == 1)? 1 : 0;


//assertions

//FIFO2 7 FIFO6 ~> FIFO9
always_comb begin
    //FIFO2
    if(!F_if.rst_n) begin
        reset_assertion: assert final(count == 0 && wr_ptr == 0 &&
                                      F_if.wr_ack == 0 && F_if.overflow == 0 &&
                                      rd_ptr == 0 && F_if.underflow == 0);
        reset_cover : cover final(count == 0 && wr_ptr == 0 &&
                                      F_if.wr_ack == 0 && F_if.overflow == 0 &&
                                      rd_ptr == 0 && F_if.underflow == 0);
    end

    //FIFO6
    if (count == 0) begin
        empty_flag_assertion: assert (F_if.empty);
        empty_flag_cover: cover (F_if.empty);
    end

    //FIFO7
    if (count == F_if.FIFO_DEPTH) begin
        full_flag_assertion: assert (F_if.full);
        full_flag_cover: cover (F_if.full);
    end
```

```
    //FIFO8
    if (count == F_if.FIFO_DEPTH-1) begin
        almostfull_flag_assertion: assert (F_if.almostfull);
        almostfull_flag_cover: cover (F_if.almostfull);
    end

    //FIFO9
    if (count == 1) begin
        almostempty_flag_assertion: assert (F_if.almostempty);
        almostempty_flag_cover: cover (F_if.almostempty);
    end
end

//FIFO3
property acknowledge;
    @(posedge F_if.clk) disable iff(!F_if.rst_n)
    (F_if.wr_en && !(F_if.full)) |=> F_if.wr_ack;
endproperty

assert property (acknowledge);
cover property (acknowledge);


//FIFO4
property overflow_attempt;
    @(posedge F_if.clk) disable iff(!F_if.rst_n)
    (F_if.wr_en && F_if.full) |=> F_if.overflow;
endproperty

assert property (overflow_attempt);
cover property (overflow_attempt);

//FIFO5
property underflow_attempt;
    @(posedge F_if.clk) disable iff(!F_if.rst_n)
    (F_if.rd_en && F_if.empty) |=> F_if.underflow;
endproperty

assert property (underflow_attempt);
cover property (underflow_attempt);

//FIFO10
property wraparound_write_pointer;
    @(posedge F_if.clk) disable iff(!F_if.rst_n)
```

```
    ((wr_ptr == F_if.FIFO_DEPTH-1) && !F_if.full && F_if.wr_en) |=> (wr_ptr ==
0);
endproperty

assert property (wraparound_write_pointer);
cover property (wraparound_write_pointer);

//FIFO11
property wraparound_read_pointer;
    @(posedge F_if.clk) disable iff(!F_if.rst_n)
    ((rd_ptr == F_if.FIFO_DEPTH-1) && !F_if.empty && F_if.rd_en) |=> (rd_ptr ==
0);
endproperty

assert property (wraparound_read_pointer);
cover property (wraparound_read_pointer);

//FIFO13
property read_pointer_boundry;
    @(posedge F_if.clk) disable iff(!F_if.rst_n)
    (rd_ptr == F_if.FIFO_DEPTH-1) |=> !(rd_ptr >= F_if.FIFO_DEPTH);
endproperty

assert property (read_pointer_boundry);
cover property (read_pointer_boundry);

//FIFO12
property write_pointer_boundry;
    @(posedge F_if.clk) disable iff(!F_if.rst_n)
    (wr_ptr == F_if.FIFO_DEPTH-1) |=> !(wr_ptr >= F_if.FIFO_DEPTH);
endproperty

assert property (write_pointer_boundry);
cover property (write_pointer_boundry);

//FIFO14
property counter_boundry;
    @(posedge F_if.clk) disable iff(!F_if.rst_n)
    (count == F_if.FIFO_DEPTH) |=> !(count > F_if.FIFO_DEPTH);
endproperty

assert property (counter_boundry);
cover property (counter_boundry);
endmodule
```

➢ The top module code:

```
module top();
    bit clk;
    always begin
        #10
        clk=~clk;
    end

    FIFO_interface F_if (clk);
    FIFO DUT (F_if);
    FIFO_tb TEST (F_if);
    FIFO_monitor MONITOR (F_if);

    //FIFO1
    always_comb begin
        if(!F_if.rst_n) begin
            reset_top_assertion: assert final(F_if.wr_ack == 0 &&
                                              F_if.overflow == 0 &&
                                              F_if.underflow == 0);
            reset_top_cover : cover final(F_if.wr_ack == 0 &&
                                          F_if.overflow == 0 &&
                                          F_if.underflow == 0);
        end
    end
endmodule
```

➢ The testbench code:

```
module FIFO_tb(FIFO_interface.TEST F_if);
    import shared_pkg::*;
    import FIFO_trans_pkg::*;
    FIFO_transaction object_rand;

    task assert_reset();
        F_if.rst_n = 0;
        @(negedge F_if.clk);
        check_reset();
        F_if.rst_n = 1;
    endtask

    task check_reset();
        bit data_ok,flags_ok;
        if(F_if.data_out == 0) begin
```

```
            data_ok = 1;
        end

        if ( F_if.wr_ack == 0 &&
            F_if.underflow == 0 && F_if.overflow == 0
            && F_if.full == 0 && F_if.empty == 1 &&
            F_if.almostfull == 0 && F_if.almostempty == 0) begin
            flags_ok = 1;
        end

        if(flags_ok & data_ok) begin
            correct_counter ++;
        end
        else begin
            $display("Error in reset");
            if (!flags_ok) begin
                $display("  Error in flags functionality");
            end
            if (!data_ok) begin
                $display("  Error in reseting data functionality");
            end
        end
    endtask

    initial begin

        object_rand = new;

        F_if.wr_en = 0;
        F_if.rd_en = 0;
        F_if.data_in = $random;
        assert_reset();

        repeat (200) begin
            assert (object_rand.randomize())
            F_if.rst_n = object_rand.rst_n_c;
            F_if.wr_en = object_rand.wr_en_c;
            F_if.rd_en = object_rand.rd_en_c;
            F_if.data_in = object_rand.data_in_c;
            @(negedge F_if.clk);
            -> F_if.start_to_sample;
        end


        finish_flg = 1;
```

```
        @(negedge F_if.clk);
        -> F_if.start_to_sample;


    end
endmodule
```

➢ The monitor code:

```
module FIFO_monitor(FIFO_interface.MONITOR F_if);
import FIFO_coverage_pkg::*;
import FIFO_trans_pkg::*;
import FIFO_scoreboard_pkg::*;
import shared_pkg::*;

FIFO_coverage F_cvr_mon = new;
FIFO_transaction F_trans_mon = new;
FIFO_scoreboard F_sb_mon = new;

initial begin
    forever begin
        //trigger
        @ F_if.start_to_sample;
        @(negedge F_if.clk);
        F_trans_mon.data_in_c = F_if.data_in;
        F_trans_mon.wr_en_c = F_if.wr_en;
        F_trans_mon.rd_en_c = F_if.rd_en;
        F_trans_mon.rst_n_c = F_if.rst_n;
        F_trans_mon.data_out_c = F_if.data_out;
        F_trans_mon.full_c = F_if.full;
        F_trans_mon.almostfull_c = F_if.almostfull;
        F_trans_mon.empty_c = F_if.empty;
        F_trans_mon.almostempty_c = F_if.almostempty;
        F_trans_mon.overflow_c = F_if.overflow;
        F_trans_mon.underflow_c = F_if.underflow;
        F_trans_mon.wr_ack_c = F_if.wr_ack;

        fork
            begin
                if(F_if.rst_n) begin
                   F_cvr_mon.sample_data(F_trans_mon);
                end
            end
            begin
```

```
                F_sb_mon.check_data(F_trans_mon);
            end
        join

        if (finish_flg) begin
            $display("Test is finished");
            $display("error count: %d & correct count:
%d",error_counter,correct_counter);
            $stop;
        end

    end
end

endmodule
```

➢ The interface code:

```
interface FIFO_interface(clk);

    parameter FIFO_DEPTH = 8,
              FIFO_WIDTH = 16;
    input clk;
    logic [FIFO_WIDTH-1:0] data_in;
    logic clk, rst_n, wr_en, rd_en;
    logic [FIFO_WIDTH-1:0] data_out;
    logic wr_ack, overflow;
    logic full, empty, almostfull, almostempty, underflow;

    event start_to_sample;


    modport DUT (
    input clk,data_in,rst_n,wr_en,rd_en,
    output data_out,wr_ack,overflow,full,
           empty,almostfull,almostempty,underflow
    );

    modport TEST(
    input clk,data_out,wr_ack,overflow,full,
           empty,almostfull,almostempty,underflow, start_to_sample,
    output data_in,rst_n,wr_en,rd_en
    );

    modport MONITOR (
```

```
    input clk,data_in,rst_n,wr_en,rd_en,
          data_out,wr_ack,overflow,full,start_to_sample,
          empty,almostfull,almostempty,underflow
    );


endinterface
```

- The packages' codes:
  - FIFO_transaction pckage:

```
package FIFO_trans_pkg;

class FIFO_transaction;

    parameter FIFO_WIDTH_TRANS = 16,
              FIFO_DEPTH_TRANS = 8;

    rand logic [FIFO_WIDTH_TRANS-1:0] data_in_c;
    rand logic rst_n_c, wr_en_c, rd_en_c;
    logic [FIFO_WIDTH_TRANS-1:0] data_out_c;
    logic wr_ack_c, overflow_c;
    logic full_c, empty_c, almostfull_c, almostempty_c, underflow_c;

    integer RD_EN_ON_DIST,WR_EN_ON_DIST;

    constraint reset_rate {
        rst_n_c dist {1:=95,0:=5};
    }

    constraint write_rate {
        wr_en_c dist {1:=WR_EN_ON_DIST,0:=(100-WR_EN_ON_DIST)};
    }

    constraint read_rate {
        rd_en_c dist {1:=RD_EN_ON_DIST,0:=(100-RD_EN_ON_DIST)};
    }

    function new (int i_r = 30 , int i_w = 70);
        this.RD_EN_ON_DIST = i_r;
        this.WR_EN_ON_DIST = i_w;
    endfunction
```

```
endclass

endpackage
```

- FIFO_coverage package:

```
package FIFO_coverage_pkg;
import FIFO_trans_pkg::*;

class FIFO_coverage;
    FIFO_transaction F_cvg_txn;

    covergroup enable_with_ops;

        //FIFO3 ~> FIFO15
        WR_eanble_CP: coverpoint F_cvg_txn.wr_en_c {
            bins on = {1};
            bins off = {0};
        }

        //FIFO3 ~> FIFO15
        RD_enable_CP: coverpoint F_cvg_txn.rd_en_c{
            bins on = {1};
            bins off = {0};
        }

        //FIFO7
        full_CP: coverpoint F_cvg_txn.full_c{
            bins on = {1};
            bins off = {0};
        }

        //FIFO8
        almostfull_CP: coverpoint F_cvg_txn.almostfull_c{
            bins on = {1};
            bins off = {0};
        }

        //FIFO6
        empty_CP: coverpoint F_cvg_txn.empty_c {
            bins on = {1};
            bins off = {0};
        }
```

```
        //FIFO9
        almostempty_CP: coverpoint F_cvg_txn.almostempty_c {
            bins on = {1};
            bins off = {0};
        }

        //FIFO3
        overflow_CP: coverpoint F_cvg_txn.overflow_c {
            bins on = {1};
            bins off = {0};
        }

        //FIFO4
        underflow_CP: coverpoint F_cvg_txn.underflow_c {
            bins on = {1};
            bins off = {0};
        }

        //FIFO3
        wr_ack_CP: coverpoint F_cvg_txn.wr_ack_c {
            bins on = {1};
            bins off = {0};
        }

        //crosses

        //FIFO7
        enable_with_full: cross WR_eanble_CP,RD_enable_CP,full_CP {
            ignore_bins x = binsof(RD_enable_CP.on) && binsof(full_CP.on);
        }

        //FIFO8
        enable_with_almostfull: cross WR_eanble_CP,RD_enable_CP,almostfull_CP;

        //FIFO6
        enable_with_empty: cross WR_eanble_CP,RD_enable_CP,empty_CP {
            ignore_bins x = binsof(WR_eanble_CP.on) && binsof(empty_CP.on);
        }

        //FIFO9
        enable_with_almostempty: cross WR_eanble_CP,RD_enable_CP,almostempty_CP;

        enable_with_overflow: cross WR_eanble_CP,RD_enable_CP,overflow_CP {
            ignore_bins x = binsof(WR_eanble_CP.off) && binsof(overflow_CP.on);
        }
```

```
        //FIFO5
        enable_with_underflow: cross WR_eanble_CP,RD_enable_CP,underflow_CP {
            ignore_bins x = binsof(RD_enable_CP.off) && binsof(underflow_CP.on);
        }

        //FIFO3
        enable_with_ack: cross WR_eanble_CP,RD_enable_CP,wr_ack_CP {
            ignore_bins x = binsof(WR_eanble_CP.off) && binsof(wr_ack_CP.on);
        }

    endgroup

    function void sample_data(FIFO_transaction F_txn);
        F_cvg_txn = F_txn;
        enable_with_ops.sample();
    endfunction

    function new ();
        enable_with_ops = new;
        F_cvg_txn = new;
    endfunction


endclass


endpackage
```

- FIFO_scoreboard package:

```
package FIFO_scoreboard_pkg;
import FIFO_trans_pkg::*;
import shared_pkg::*;


class FIFO_scoreboard;

    parameter FIFO_WIDTH_SB = 16,
              FIFO_DEPTH_SB = 8;

    //FIFO_transaction F_sb_test;
    logic [FIFO_WIDTH_SB-1:0] data_out_ref;
    logic wr_ack_ref, overflow_ref;
    logic full_ref, empty_ref, almostfull_ref, almostempty_ref, underflow_ref;
```

```
    logic [FIFO_WIDTH_SB-1:0] data_queue [$];

    // bits to determine to do write operation and read operation in refernce
model
    bit do_wr,do_rd ;

    task check_data(FIFO_transaction F_sb_test);
        reference_model(F_sb_test);
        //check data out
        if(F_sb_test.data_out_c === data_out_ref) begin
            correct_counter = correct_counter + 1;
        end
        else begin
            $display("Error: data out exp: %h , got:
%h",data_out_ref,F_sb_test.data_out_c);
            error_counter = error_counter + 1;
        end
    endtask

     //FIFO15
    task reference_model(FIFO_transaction F_sb_ref);
        if(!F_sb_ref.rst_n_c) begin
            data_queue.delete();
            wr_ack_ref = 0;
            underflow_ref = 0;
            overflow_ref = 0;
        end
        else begin
            //determine to do write and read operations or not
            if(F_sb_ref.wr_en_c && F_sb_ref.rd_en_c) begin
                //if empty write only
                if(data_queue.size() == 0) begin
                    do_wr = 1;
                    do_rd = 0;
                    underflow_ref = 1;
                end
                //if full read only
                else if (data_queue.size() == FIFO_DEPTH_SB) begin
                    do_wr = 0;
                    do_rd = 1;
                    overflow_ref = 1;
                end
                else begin
                    do_wr = 1;
```

```
                do_rd = 1;
            end
        end
        else begin
            do_wr = F_sb_ref.wr_en_c;
            do_rd = F_sb_ref.rd_en_c;
        end

        //write operation
        if(do_wr) begin
            if (data_queue.size() < FIFO_DEPTH_SB ) begin
                data_queue.push_back(F_sb_ref.data_in_c);
                wr_ack_ref = 1;
                overflow_ref = 0;
            end
            else begin
                overflow_ref = 1;
                wr_ack_ref = 0;
            end
        end
        else begin
            wr_ack_ref = 0;
        end

        //read operation
         if(do_rd) begin
            if (data_queue.size() > 0 ) begin
                data_out_ref = data_queue.pop_front();
                underflow_ref = 0;
            end
            else begin
                underflow_ref = 1;
            end
        end

        //flags
        full_ref = (data_queue.size() == FIFO_DEPTH_SB);
        empty_ref = (data_queue.size() == 0);
        almostempty_ref = (data_queue.size() == 1);
        almostfull_ref = (data_queue.size() == (FIFO_DEPTH_SB-1));
    end
endtask

endclass
endpackage
```

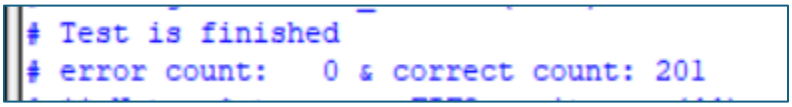
- Shared_pkg package:

```
package shared_pkg;
    bit [8:0] correct_counter , error_counter;
    bit finish_flg;
endpackage
```

The do file:

vlib work
vlog *v +cover
vsim -voptargs=+acc top -cover
run 0
add wave *
add wave -position insertpoint \
sim:/top/MONITOR/F_sb_mon \
sim:/top/MONITOR/F_trans_mon
add wave /top/reset_top_assertion /top/DUT/reset_assertion
/top/DUT/empty_flag_assertion /top/DUT/full_flag_assertion
/top/DUT/almostfull_flag_assertion /top/DUT/almostempty_flag_assertion
/top/DUT/assert__acknowledge /top/DUT/assert__overflow_attempt
/top/DUT/assert__underflow_attempt /top/DUT/assert__wraparound_write_pointer
/top/DUT/assert__wraparound_read_pointer /top/DUT/assert__read_pointer_boundry
/top/DUT/assert__write_pointer_boundry /top/DUT/assert__counter_boundry
coverage save FIFO_top.ucdb -onexit
run -all
vcover report FIFO_top.ucdb -details -annotate -all -output coverage_rpt_FIFO.txt

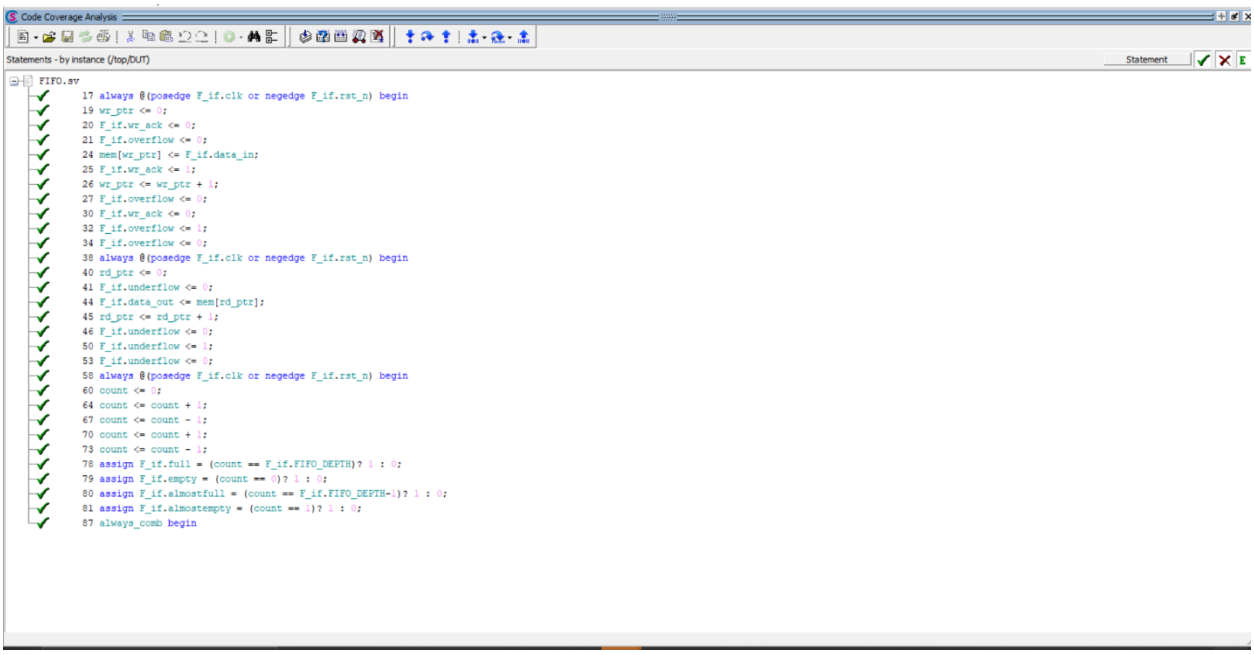
The transcript snippet:



The correct count shows how many times that the data out is checked not the assertions.

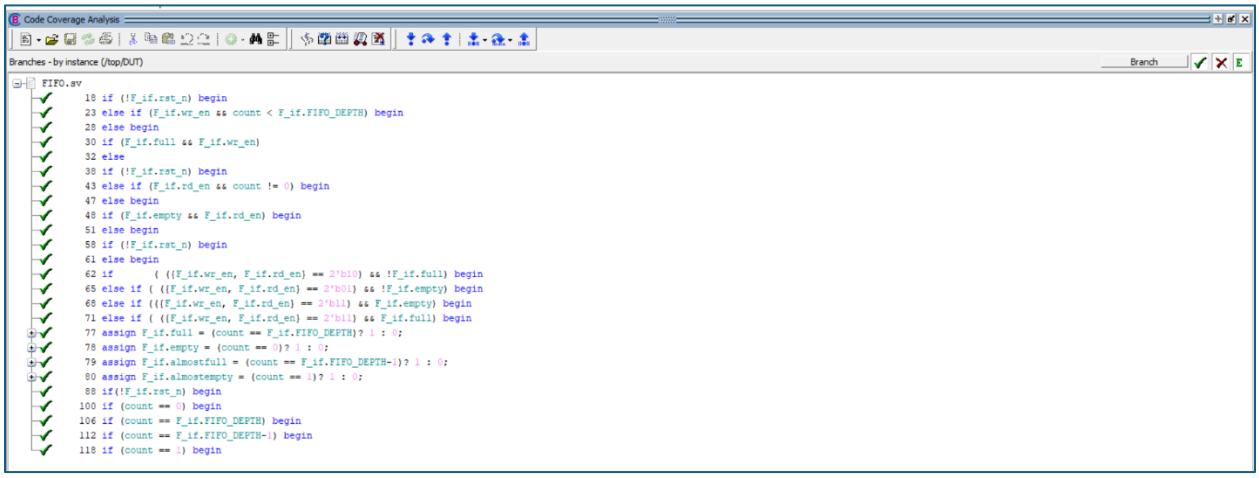
The code coverage report:

➢ Statement:



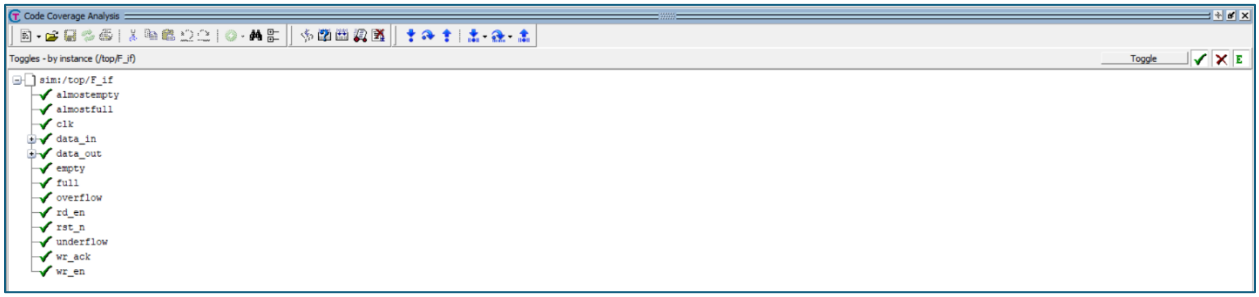

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      29        29         0   100.00%

================================Statement Details================================

Statement Coverage for instance /top/DUT --
NOTE: The modification timestamp for source file 'FIFO.sv' has been altered since compilation.

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File FIFO.sv
    8                                                module FIFO(FIFO_interface.DUT F_if);

    9

    10                                               localparam max_fifo_addr = $clog2(F_if.FIFO_DEPTH);

    11

    12                                               reg [F_if.FIFO_WIDTH-1:0] mem [F_if.FIFO_DEPTH-1:0];

    13

    14                                               reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;

    15                                               reg [max_fifo_addr:0] count;

    16

    17              1                         211    always @(posedge F_if.clk or negedge F_if.rst_n) begin

    18                                                  if (!F_if.rst_n) begin

    19              1                          19              wr_ptr <= 0;

    20              1                          19              F_if.wr_ack <= 0;

    21              1                          19              F_if.overflow <= 0;

    22                                                  end
```

➢ Branch:



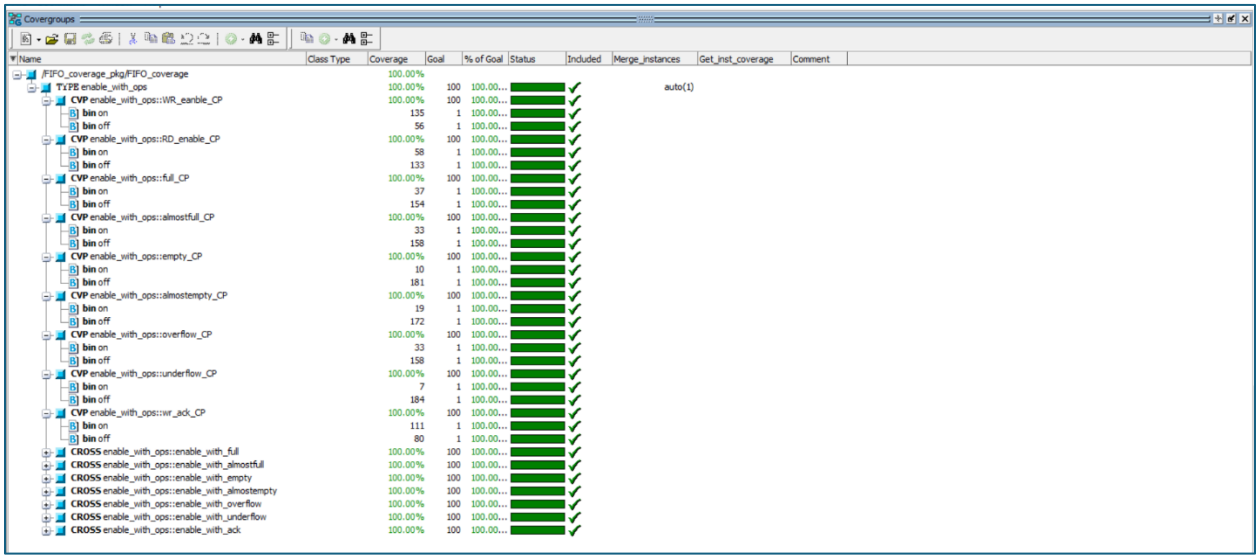

```
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        35        35         0   100.00%

================================Branch Details================================

Branch Coverage for instance /top/DUT
NOTE: The modification timestamp for source file 'FIFO.sv' has been altered since compilation.

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File FIFO.sv
------------------------------------IF Branch------------------------------------
    18                                       211     Count coming in to IF
    18              1                         19        if (!F_if.rst_n) begin

    23              1                        111        else if (F_if.wr_en && count < F_if.FIFO_DEPTH) begin

    28              1                         81        else begin

Branch totals: 3 hits of 3 branches = 100.00%

------------------------------------IF Branch------------------------------------
    30                                        81     Count coming in to IF
    30              1                         24                if (F_if.full && F_if.wr_en)

    32              1                         57                else

Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    38                                       211     Count coming in to IF
    38              1                         19        if (!F_if.rst_n) begin

    43              1                         53        else if (F_if.rd_en && count != 0) begin

    47              1                        139        else begin

Branch totals: 3 hits of 3 branches = 100.00%

------------------------------------IF Branch------------------------------------
    48                                       139     Count coming in to IF
```

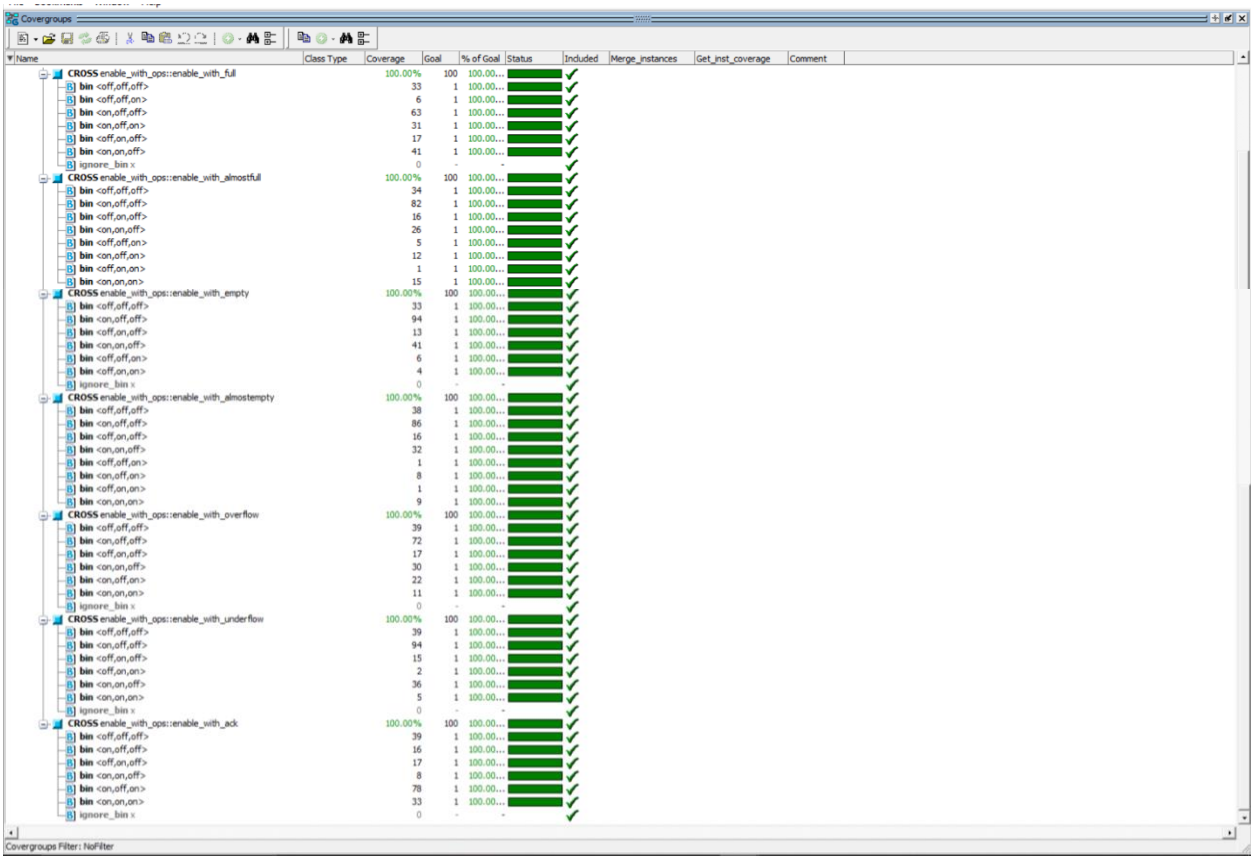
➢ Toggle:




```
================================================================================
=== Instance: /top/F_if
=== Design Unit: work.FIFO_interface
================================================================================
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         86        86         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /top/F_if --

                                          Node      1H->0L      0L->1H  "Coverage"
                                          ----------------------------------------
                                    almostempty           1           1      100.00
                                     almostfull           1           1      100.00
                                            clk           1           1      100.00
                                   data_in[15-0]          1           1      100.00
                                  data_out[15-0]          1           1      100.00
                                           empty          1           1      100.00
                                            full          1           1      100.00
                                        overflow          1           1      100.00
                                           rd_en          1           1      100.00
                                           rst_n          1           1      100.00
                                       underflow          1           1      100.00
                                          wr_ack          1           1      100.00
                                           wr_en          1           1      100.00

Total Node Count     =         43
Toggled Node Count   =         43
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (86 of 86 bins)
```

The functional coverage:

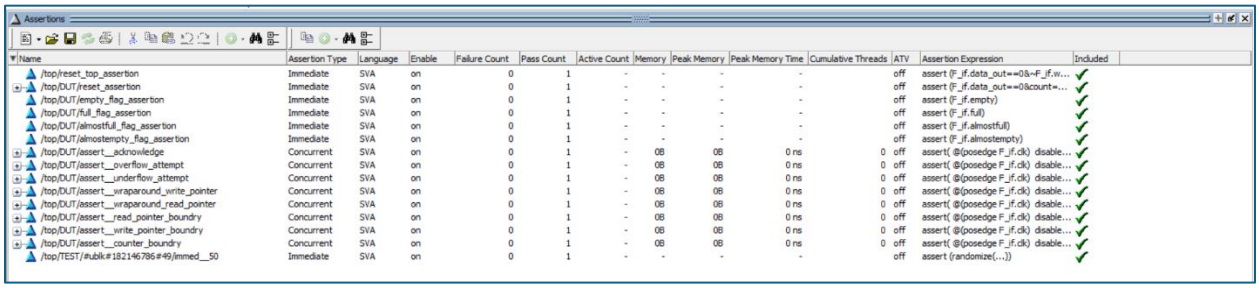


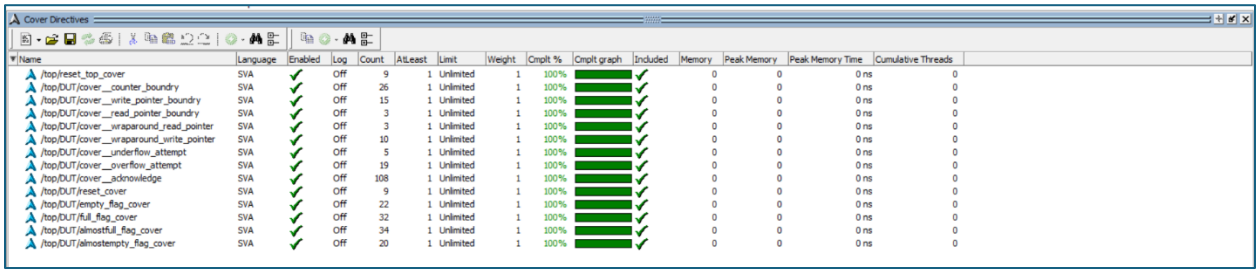


Comment: the ignored cross bins are the bins that can't be happen in normal conditions or doesn't have meaning. The ignored pins are:

- Empty on with wr_en on as empty is combinational so when see write operation it changed immediately and there is no meaning to see empty flag with write operation.
- Full on with rd_en on as empty is combinational so when see read operation it changed immediately and there is no meaning to see full flag with read operation.
- The wr_ack on with wr_en off as wr_ack is defined to sure that the write operation is done so it can't be achieved.
- The underflow on with rd_en of as underflow is defined signal to detect attempt to read for empty FIFO so it can't be achieved.
- The overflow on with wr_en of as overflow is defined signal to detect attempt to write for full FIFO so it can't be achieved.

The assertion coverage:



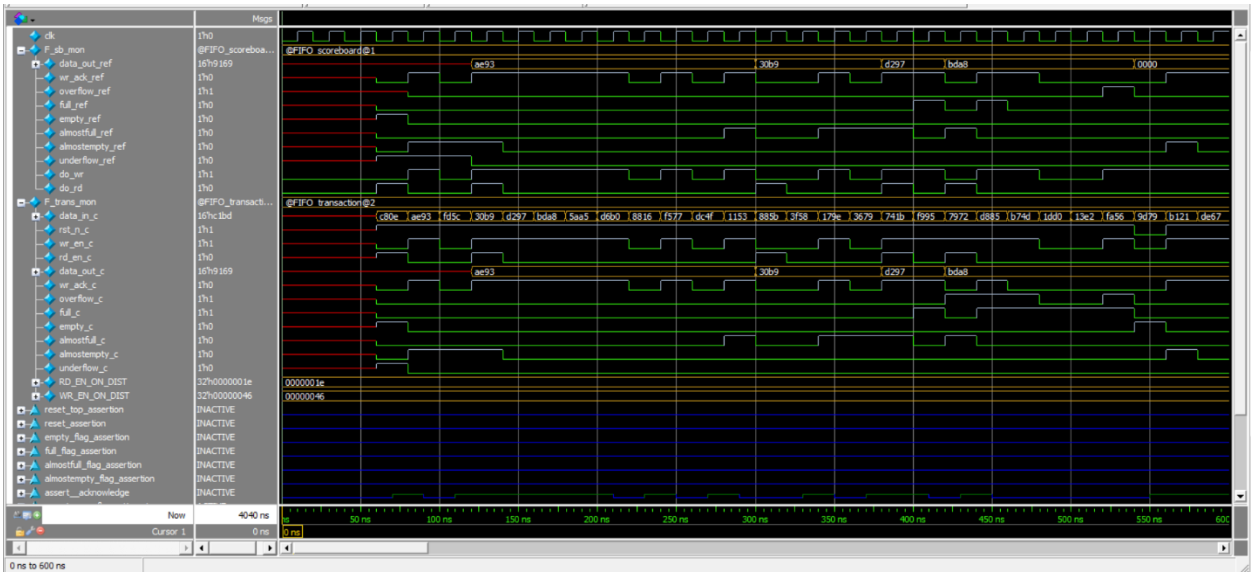


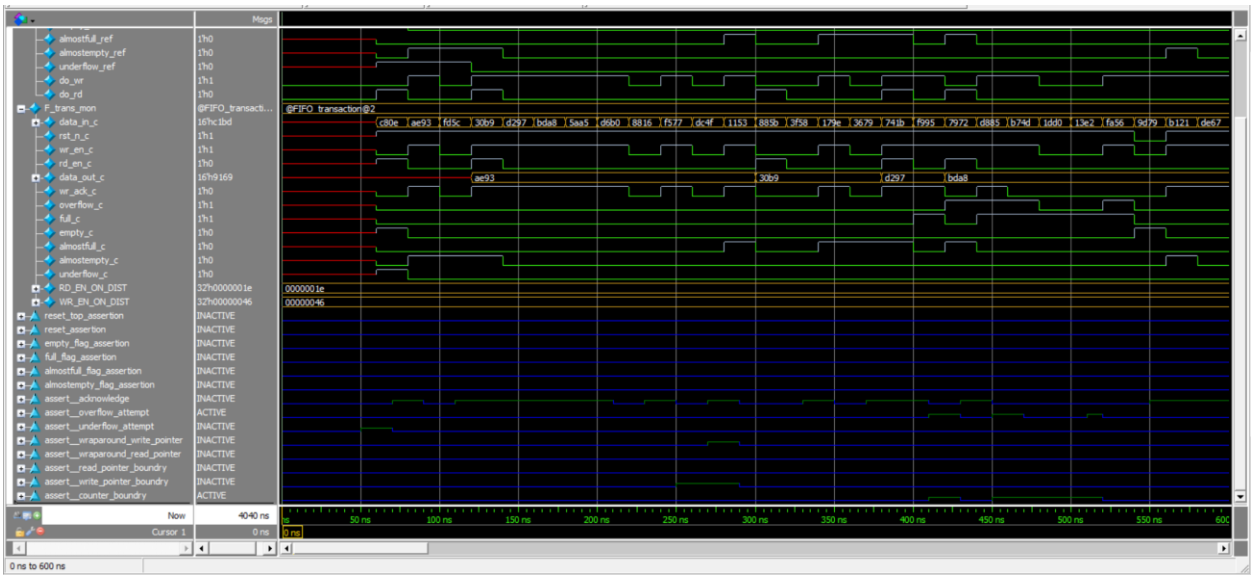
```
DIRECTIVE COVERAGE:
-----------------------------------------------------------------------------------------
Name                                  Design Design   Lang File(Line)      Hits Status
                                      Unit   UnitType
-----------------------------------------------------------------------------------------
/top/reset_top_cover                  top    Verilog  SVA  FIFO_t.sv(18)      9 Covered
/top/DUT/cover__counter_boundry       FIFO   Verilog  SVA  FIFO.sv(178)      26 Covered
/top/DUT/cover__write_pointer_boundry FIFO   Verilog  SVA  FIFO.sv(170)      15 Covered
/top/DUT/cover__read_pointer_boundry  FIFO   Verilog  SVA  FIFO.sv(162)       3 Covered
/top/DUT/cover__wraparound_read_pointer FIFO Verilog  SVA  FIFO.sv(154)       3 Covered
/top/DUT/cover__wraparound_write_pointer FIFO Verilog SVA  FIFO.sv(146)      10 Covered
/top/DUT/cover__underflow_attempt     FIFO   Verilog  SVA  FIFO.sv(138)       5 Covered
/top/DUT/cover__overflow_attempt      FIFO   Verilog  SVA  FIFO.sv(130)      19 Covered
/top/DUT/cover__acknowledge           FIFO   Verilog  SVA  FIFO.sv(122)     108 Covered
/top/DUT/reset_cover                  FIFO   Verilog  SVA  FIFO.sv(91)        9 Covered
/top/DUT/empty_flag_cover             FIFO   Verilog  SVA  FIFO.sv(98)       22 Covered
/top/DUT/full_flag_cover              FIFO   Verilog  SVA  FIFO.sv(102)      32 Covered
/top/DUT/almostfull_flag_cover        FIFO   Verilog  SVA  FIFO.sv(107)      34 Covered
/top/DUT/almostempty_flag_cover       FIFO   Verilog  SVA  FIFO.sv(112)      20 Covered

TOTAL DIRECTIVE COVERAGE: 100.00%  COVERS: 14
```

```
ASSERTION RESULTS:
---------------------------------------------------------------------
Name                 File(Line)                   Failure      Pass
                                                  Count        Count
---------------------------------------------------------------------
/top/reset_top_assertion
                     FIFO_t.sv(14)                      0          1
/top/DUT/assert__counter_boundry
                     FIFO.sv(177)                       0          1
/top/DUT/assert__write_pointer_boundry
                     FIFO.sv(169)                       0          1
/top/DUT/assert__read_pointer_boundry
                     FIFO.sv(161)                       0          1
/top/DUT/assert__wraparound_read_pointer
                     FIFO.sv(153)                       0          1
/top/DUT/assert__wraparound_write_pointer
                     FIFO.sv(145)                       0          1
/top/DUT/assert__underflow_attempt
                     FIFO.sv(137)                       0          1
/top/DUT/assert__overflow_attempt
                     FIFO.sv(129)                       0          1
/top/DUT/assert__acknowledge
                     FIFO.sv(121)                       0          1
/top/DUT/reset_assertion
                     FIFO.sv(87)                        0          1
/top/DUT/empty_flag_assertion
                     FIFO.sv(97)                        0          1
/top/DUT/full_flag_assertion
                     FIFO.sv(101)                       0          1
/top/DUT/almostfull_flag_assertion
                     FIFO.sv(106)                       0          1
/top/DUT/almostempty_flag_assertion
                     FIFO.sv(111)                       0          1
/top/TEST/#ublk#182146786#49/immed__50
                     FIFO_tb.sv(50)                     0          1
```

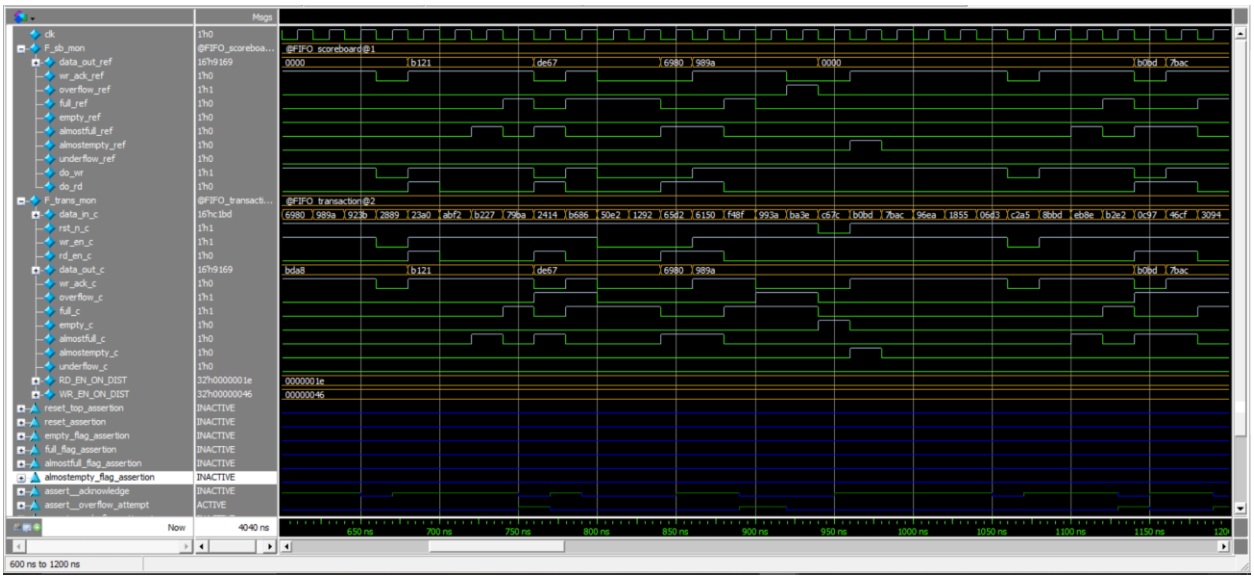
The waveform snippets:

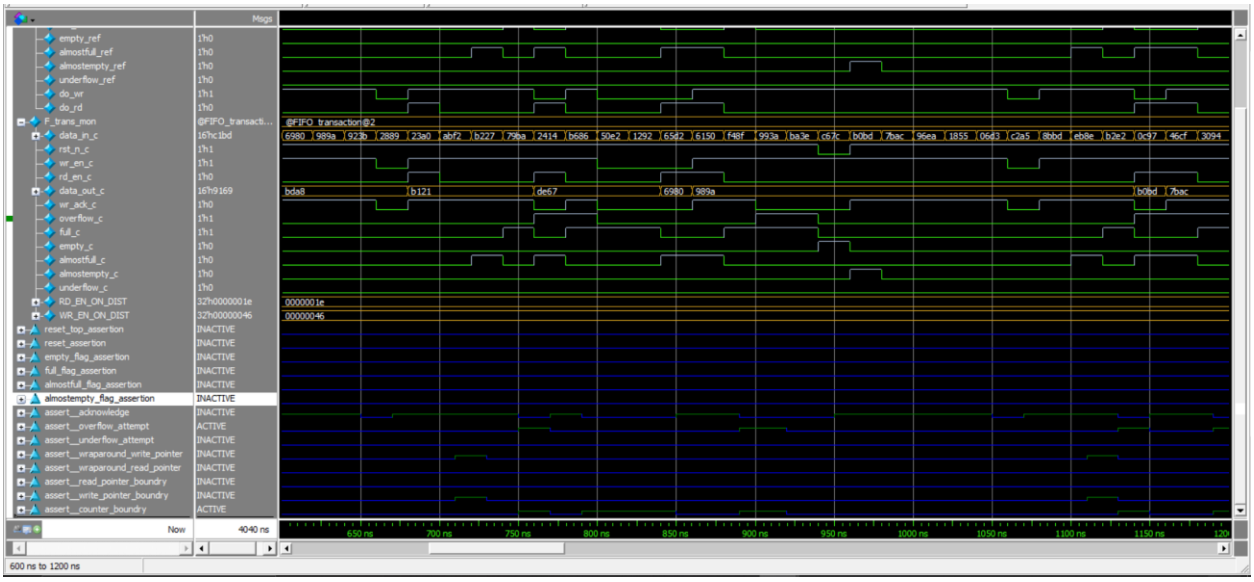
0 ~> 600ns



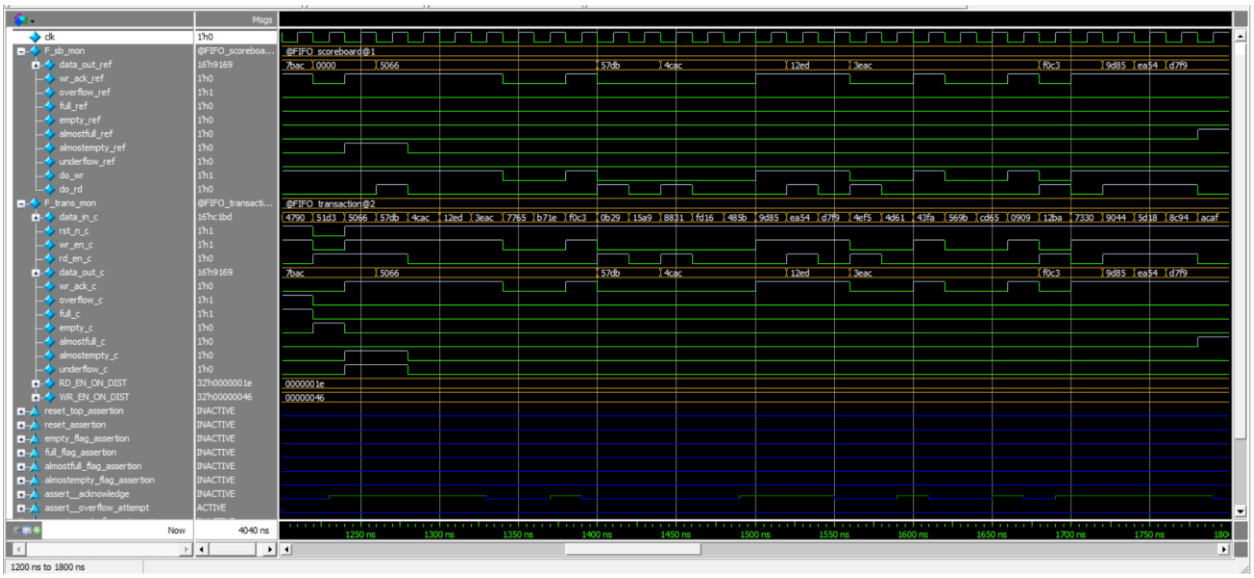


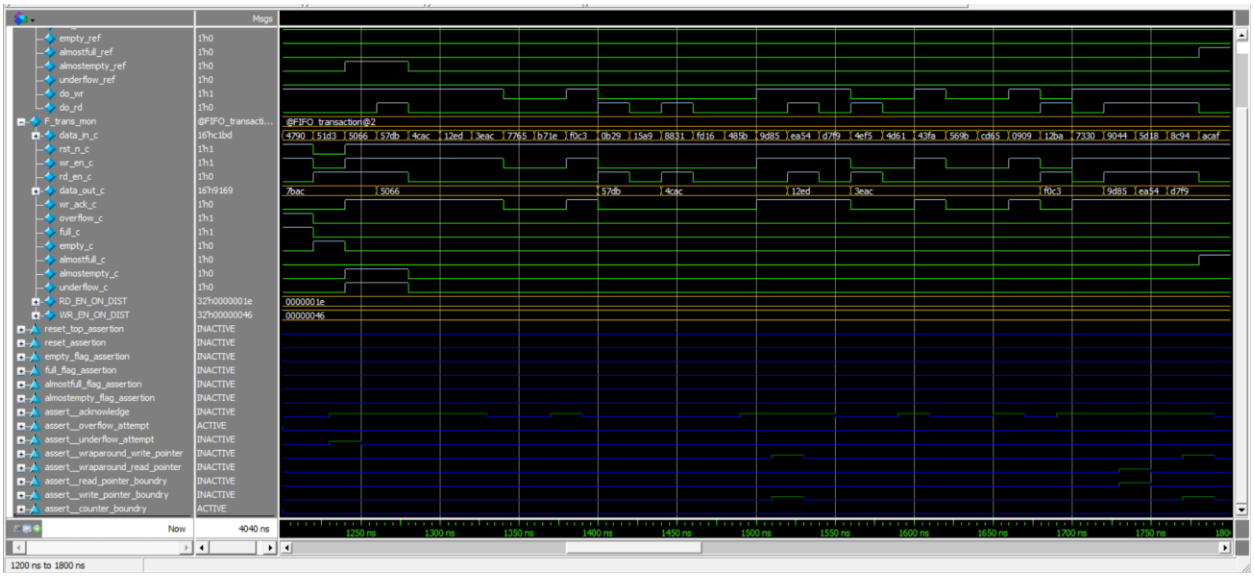
600ns ~> 1200ns



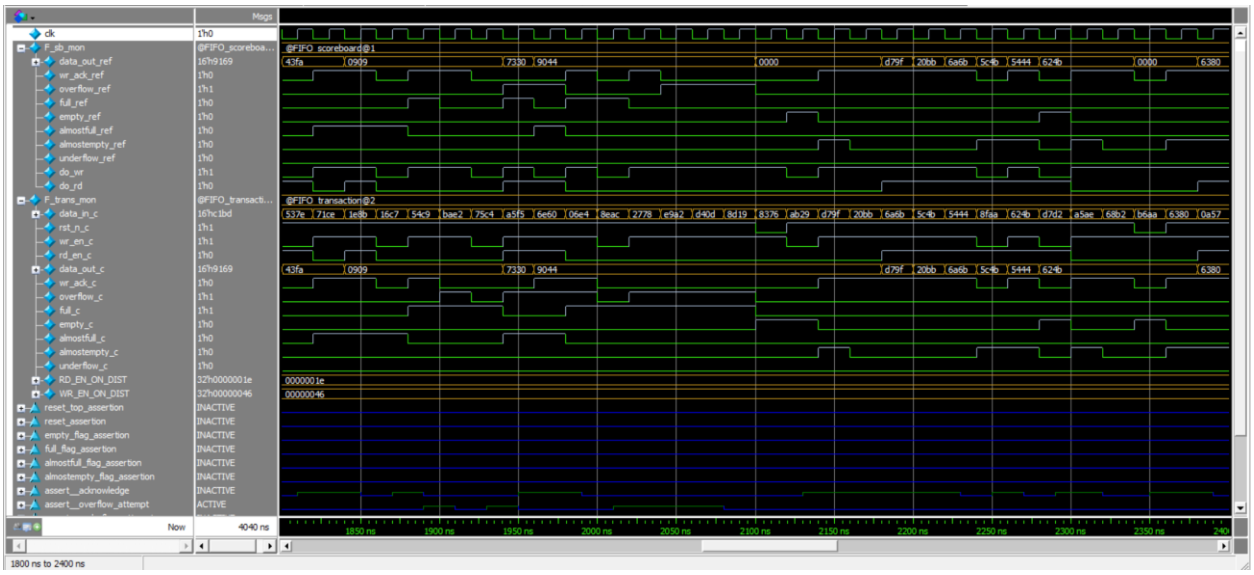


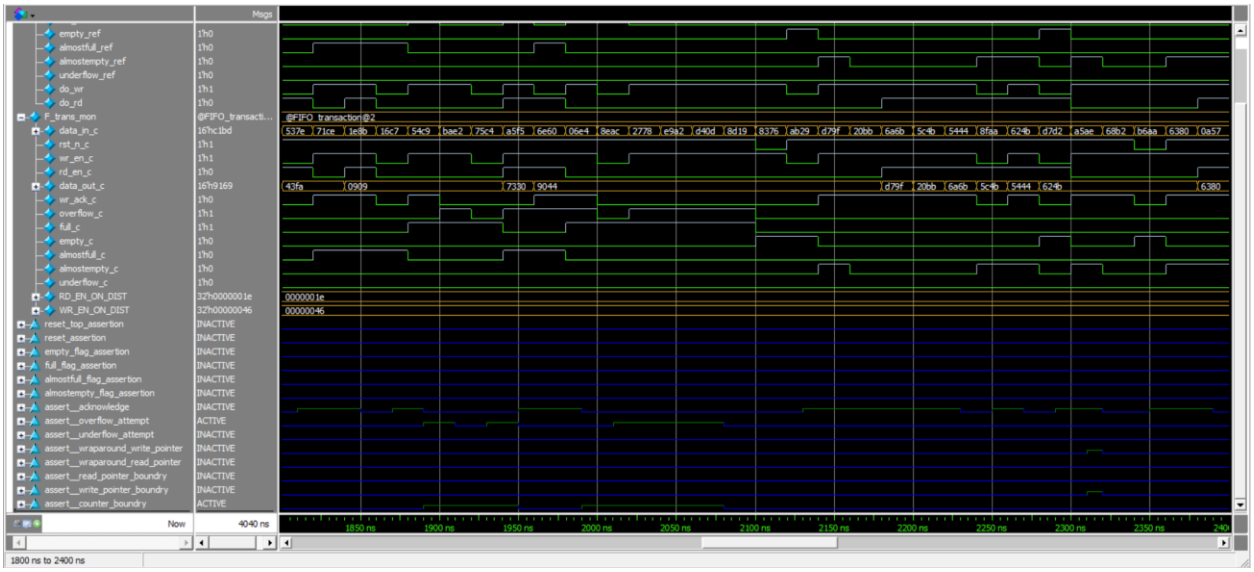
1200ns ~> 1800ns



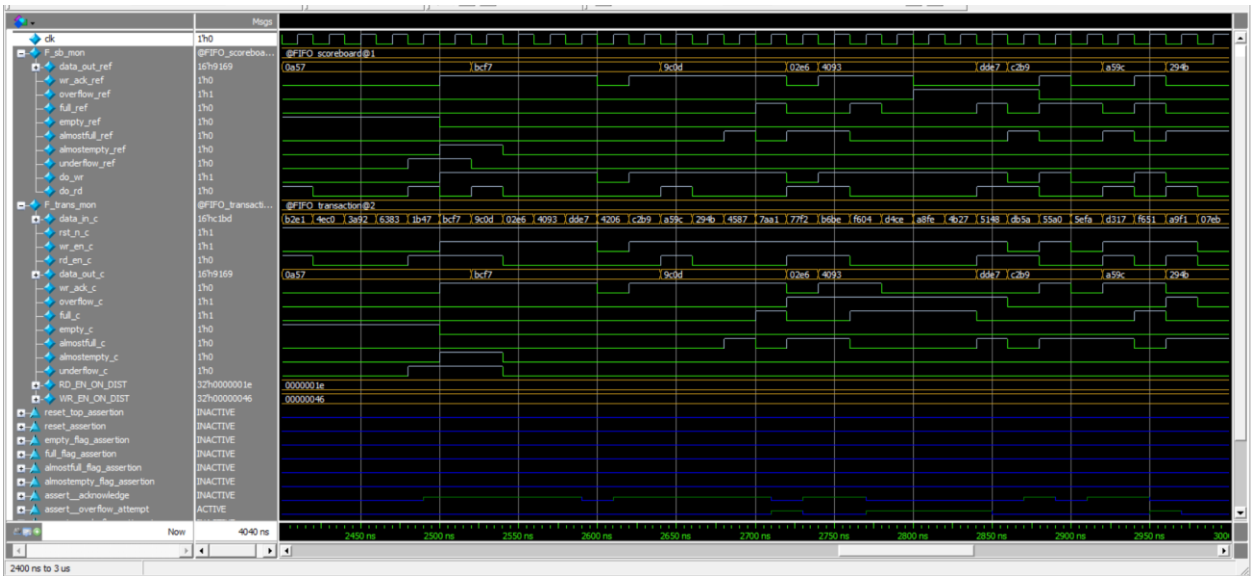


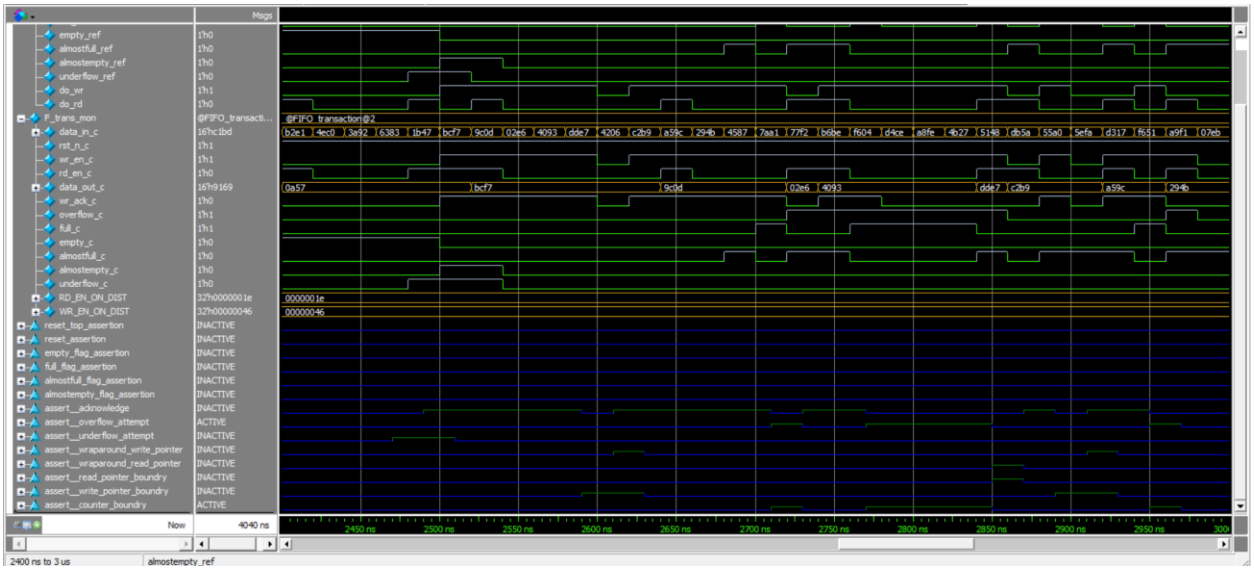
1800ns ~> 2400ns



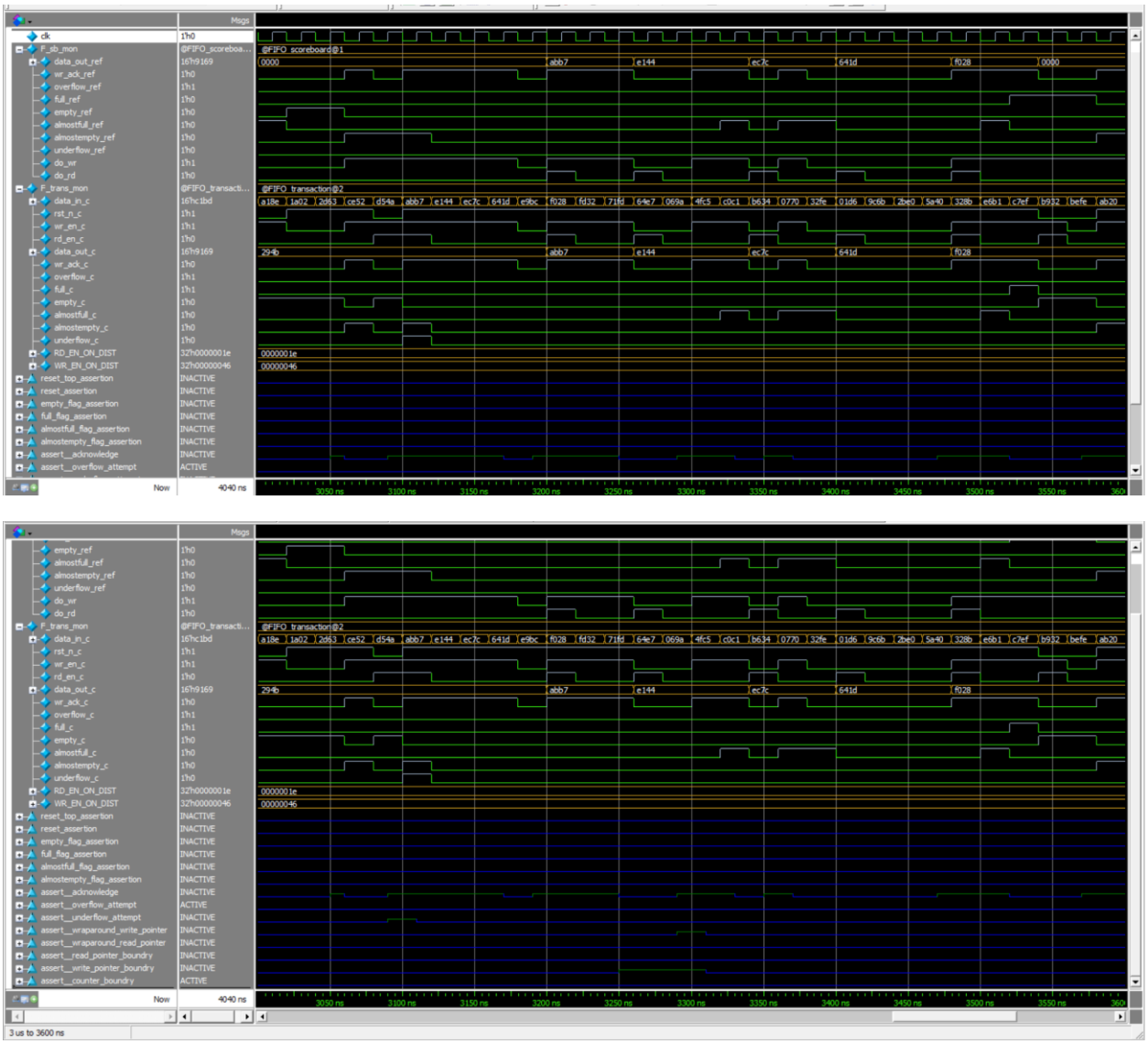


2400ns ~> 3000ns

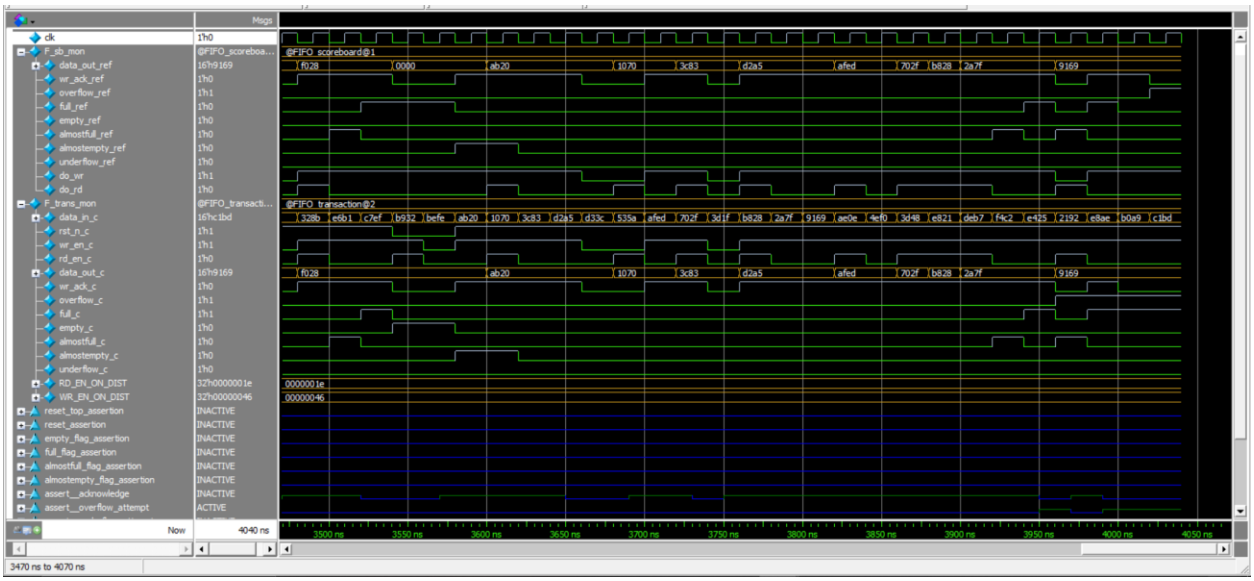


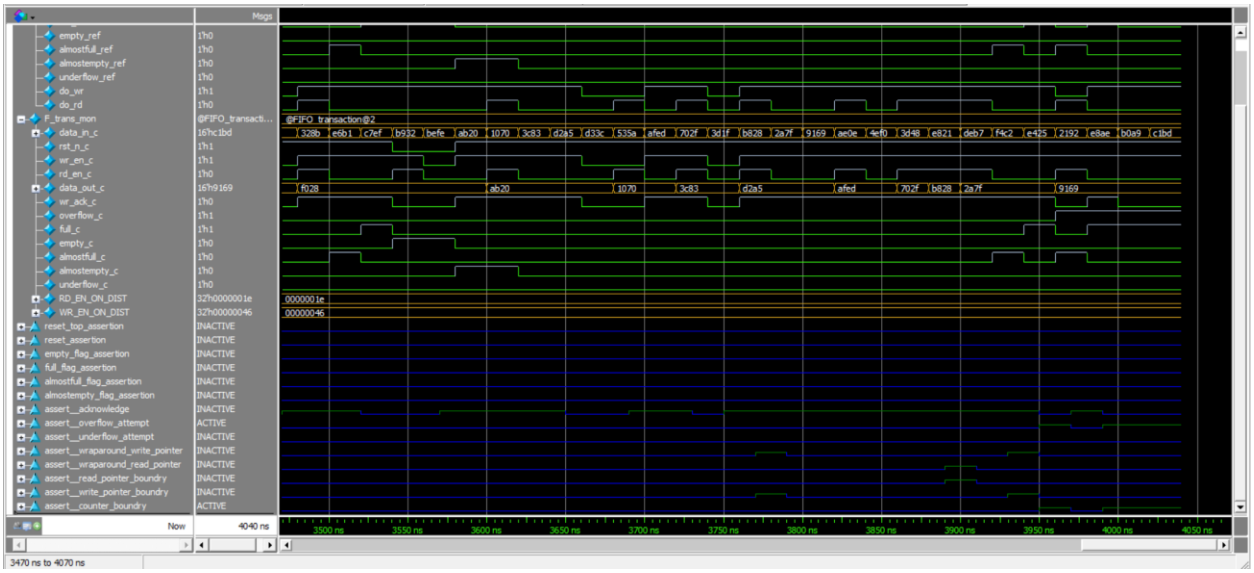


3000ns ~> 3600ns



3450ns ~> end


clk
F_sb_mon
data_out_ref
wr_ack_ref
overflow_ref
full_ref
empty_ref
almostfull_ref
almostempty_ref
underflow_ref
do_wr
do_rd
F_trans_mon
data_in_c
rst_n_c
wr_en_c
rd_en_c
data_out_c
wr_ack_c
overflow_c
full_c
empty_c
almostfull_c
almostempty_c
underflow_c
RD_EN_ON_DIST
WR_EN_ON_DIST
reset_top_assertion
reset_assertion
empty_flag_assertion
full_flag_assertion
almostfull_flag_assertion
almostempty_flag_assertion
assert__acknowledge
assert__overflow_attempt
Now
4040 ns
3470 ns to 4070 ns


empty_ref
almostfull_ref
almostempty_ref
underflow_ref
do_wr
do_rd
F_trans_mon
data_in_c
rst_n_c
wr_en_c
rd_en_c
data_out_c
wr_ack_c
overflow_c
full_c
empty_c
almostfull_c
almostempty_c
underflow_c
RD_EN_ON_DIST
WR_EN_ON_DIST
reset_top_assertion
reset_assertion
empty_flag_assertion
full_flag_assertion
almostfull_flag_assertion
almostempty_flag_assertion
assert__acknowledge
assert__overflow_attempt
assert__underflow_attempt
assert__wraparound_write_pointer
assert__wraparound_read_pointer
assert__read_pointer_boundry
assert__write_pointer_boundry
assert__counter_boundry
Now
4040 ns
3470 ns to 4070 ns